

计算大点数的快速卷积

方法一：分段、重叠相加法（Overlap-Add Method）

1. 现实的残酷：为什么要分段？

想象你正在处理一段无限长的音频流（或者长度极其恐怖，比如几百万个点）。

你的滤波器 $h(n)$ 只有 $M = 100$ 个点。

- **如果直接算线性卷积：** 你必须等这首几百万个点的歌全部播放完，才能开始算。这叫“无限延迟”，实时系统直接崩溃。
- **如果想用 FFT 加速：** 你得做一个几百万点的巨型 FFT，芯片的内存当场爆炸。

工程师的破解之道：流水线“切糕法”。

既然一口吃不下，那我们就把无限长的信号 $x(n)$ ，切成一段一段固定长度 L 的“小切糕”，来一段，处理一段，最后再拼起来！

2. 物理直觉流水线：重叠相加法的“三步走”

第一步：无缝切片（拆解输入）

也就是 PPT 里的公式 $x_i(n)$ 。

我们把无限长、绵延不断的 $x(n)$ ，**毫不重叠地**切成一段段长度为 L 的小片段。

比如 $L = 1000$ ：

- 第 0 段：包含 $n = 0 \sim 999$ 的信号。
- 第 1 段：包含 $n = 1000 \sim 1999$ 的信号。
- 第 2 段：包含 $n = 2000 \sim 2999$ 的信号。它们就像排着队进厂的原材料，首尾相接，但绝不重叠。

第二步：膨胀加工（用 FFT 算分段卷积）

也就是 PPT 里的 $y_i(n) = x_i(n) * h(n)$ 。

这一步是核心！我们把每一段切好的信号（长度 L ），分别送去和滤波器 $h(n)$ （长度 M ）做线性卷积。

- **物理法则的惩罚（膨胀效应）：** 根据你之前烂熟于心的定律，出来的结果 $y_i(n)$ ，它的长度必然会“膨胀”变成 $L + M - 1$ ！

- **FFT 开挂加速：**为了算得快，我们不用老古董的线性卷积公式，而是转到频域用 FFT。为了保证这个膨胀的结果不发生“时空撞车（时域混叠）”，我们开辟的环形宇宙（DFT 点数 N ）必须满足 $N \geq L + M - 1$ 。
- 为了迎合计算机的脾气（基二 FFT），我们会把 N 往上凑到最近的 2 的幂次方（比如 256、512、1024）。

第三步：重叠相加（收拾膨胀的残局）

这是这个算法名字的由来！

你想想，原材料进厂的时候，每隔 L 个点进来一截。

但是出厂的时候，每一截产物都膨胀到了 $L + M - 1$ 个点！

这就意味着：“前一段的尾巴”一定会“越界”跑进“后一段的地盘”里去！

越界了多少个点？

$$(L + M - 1) - L = M - 1$$

终极拼图（也就是 PPT 最后那个相加公式）：

既然卷积是一个完全线性的系统，上帝法则是允许我们把分开算的结果直接加起来的。

所以，在输出端：

- 第 0 段的前 L 个点是绝对干净的。
- 第 0 段的最后 $M - 1$ 个点（越界的尾巴），必须和 第 1 段的开头 $M - 1$ 个点，**在物理时空上对齐，然后强行加在一起！**
- 第 1 段的最后 $M - 1$ 个点，又必须加到 第 2 段的开头.....

以此类推，就像砌砖一样，每一块砖的尾巴都搭在下一块砖的头上（搭接长度为 $M - 1$ ）。把这些重叠的部分统统加起来，得到的就是和“一开始不切片直接算”**完全一模一样的完美连续信号！**

3.核心考点提炼（期末必考）：

1. **为什么会有重叠？** 因为输入是按 L 隔开的，输出长度是 $L + M - 1$ ，“产出比进料长”，必然往后溢出。
2. **重叠了多少点？** 永远是 $M - 1$ 个点（滤波器的长度减一）。
3. **FFT 的点数 N 怎么选？** 必须满足 $N \geq L + M - 1$ ，且最好是 2 的整数幂。这保证了在用 FFT 算每一小段的时候，**内部绝对不会发生圆周混叠。**
4. **工程设计题套路：** 题目通常会给你 M （滤波器已知），给你硬件限制的 FFT 最大点数 N （比如限制只能做 1024 点 FFT），问你最优的切片长度 L 是多少？
 - **秒杀公式：** 极大化利用内存，令 $L + M - 1 \leq N$ ，所以最优切片长度 $L = N - M + 1$ 。

4.例题

例 已知序列 $x[n]=n+2, 0 \leq n \leq 12$, $h[n]=\{1, 2, 1\}$ 试利用重叠相加法计算线性卷积, 取 $L=5$ 。

解: 重叠相加法

$$x_1[n]=\{2, 3, 4, 5, 6\} \quad x_2[n]=\{7, 8, 9, 10, 11\} \quad x_3[n]=\{12, 13, 14\}$$

$$y_1[n]=\{2, 7, 12, 16, 20, 17, 6\}$$

$$y_2[n]=\{7, 22, 32, 36, 40, 32, 11\}$$

$$y_3[n]=\{12, 37, 52, 41, 14\}$$

$$y[n]=\{2, 7, 12, 16, 20, 24, 28, 32, 36, 40, 44, 48, 52, 41, 14\}$$

方法二：分段、重叠保留法 (Overlap-Save)

1. 两大门派的底层哲学对决

- 重叠相加法 (Overlap-Add):
 - 输入端：极其干净。每次老老实实切 L 个新数据进去，互不重叠。
 - 输出端：极其麻烦。算出来的结果膨胀了，尾巴越界了，必须得用加法器把尾巴和下一段的头加起来。
- **重叠保留法 (Overlap-Save):
 - 输入端：极其贪婪（重叠）。每次切数据，不仅要 L 个新的，还要把上一段最后的 $M-1$ 个旧数据生拉硬拽过来，强行凑成 $N = L + M - 1$ 个点一起送进去算。
 - 输出端：极其省事（直接扔）。算完之后，发现头部有 $M-1$ 个点是“被污染的垃圾数据”，直接一刀砍掉舍弃！剩下的 L 个点绝对完美，直接跟下一段拼在一起就行，完全不需要做加法！

2. 为什么前 $M-1$ 个点是“垃圾”？（知识点终极闭环）

这就必须祭出我们之前反复强调的“时域混叠（火车追尾）”定律了！

这门功夫的核心命门：“如果用 DFT 实现圆周卷积……其前 $(M-1)$ 个点的值不等于线性卷积值，必须舍去。”

为什么偏偏是前 $M - 1$ 个点出错？

1. **强行开辟的环形宇宙：** 你的输入信号这次被你凑成了 N 个点（旧的 $M - 1 +$ 新的 L ），你的滤波器还是 M 个点。
2. **真实的线性卷积长度（真实的火车长度）：** $N + M - 1$ 。
3. **你给的环形跑道长度：** 只有 N ！
4. **灾难发生：** 火车长度 $(N + M - 1)$ 明显大于跑道长度 (N) 。超出去的部分是多少？刚好是 $M - 1$ 节车厢！
5. **时空撞车：** 这超出去的 $M - 1$ 节尾巴，会穿越时空，**死死地撞在跑道最前面的 $M - 1$ 个位置上！**

所以，经过 FFT 圆周卷积吐出来的数据里，最前面的 $M - 1$ 个点，是被尾巴撞毁的残骸！它们是严重混叠的错误数据。

但是，后面的 L 个点，因为跑道足够长没有被撞到，所以它们是极其纯净的、完全等价于真实线性卷积的完美数据！

处理方法极其暴力：把前面撞烂的 $M - 1$ 个点直接扔进垃圾桶，保留后面完美的 L 个点。这就是“重叠保留法”名字的由来！

3.考场绝杀总结

这两兄弟经常放在一起考概念辨析，你只需要记住这个“跷跷板”：

- **Overlap-Add (相加法)：** 不重叠切片 $\rightarrow$ 算出来尾巴变长 $\rightarrow$ 把长出来的尾巴加到下一段头上。
- **Overlap-Save (保留法)：** 重叠切片 (偷拿上局的牌) $\rightarrow$ 算出来头被混叠撞烂 $\rightarrow$ 把烂掉的头扔掉，剩下的保留拼接。

在真实的芯片设计（比如 FPGA 做音频处理）里，工程师实际上更喜欢用你发的这个“重叠保留法”。因为在硬件电路里，直接“扔掉数据、拼接数据”就是拨动一下寻址指针的事，而“做加法”却需要额外去占用极其宝贵的加法器（Adder）和缓冲寄存器。省去加法，就是省去了真金白银的硬件成本！

4.例题

例 已知序列 $x[n]=n+2, 0 \leq n \leq 12$, $h[n]=\{1,2,1\}$ 试利用重叠保留法计算线性卷积, 取 $L=5$ 。

解: 重叠保留法

$$x_1[k]=\{0, 0, 2, 3, 4\} \quad x_2[k]=\{3, 4, 5, 6, 7\} \quad x_3[k]=\{6, 7, 8, 9, 10\}$$

$$x_4[k]=\{9, 10, 11, 12, 13\} \quad x_5[k]=\{12, 13, 14, 0, 0\}$$

$$y_1[k]=x_1[k] \otimes h[k]=\{11, 4, 2, 7, 12\}$$

$$y_2[k]=x_2[k] \otimes h[k]=\{23, 17, 16, 20, 24\}$$

$$y_3[k]=x_3[k] \otimes h[k]=\{35, 29, 28, 32, 36\}$$

$$y_4[k]=x_4[k] \otimes h[k]=\{47, 41, 40, 44, 48\}$$

$$y_5[k]=x_5[k] \otimes h[k]=\{12, 37, 52, 41, 14\}$$

$$y[k]=\{2, 7, 12, 16, 20, 24, 28, 32, 36, 40, 44, 48, 52, 41, 14\}$$